Semantic Caching for RAG/LLM Pipelines — Correctness vs. Cost Tradeoffs Under Controlled Paraphrase Variation

Garv Jain
Department of Computer Science Engineering
Poornima Institute of Engineering and Technology
Jaipur, Rajasthan

Ms. Khushboo Dadhich
Assistant Professor
Department of Computer Science Engineering
Poornima Institute of Engineering and Technology
Jaipur, Rajasthan

Abstract

Semantic caching can reduce inference cost by reusing responses for semantically similar queries, but a query can be close in embedding space and still require a different answer. We evaluate this risk on a fixed 700-query workload derived from 100 NQ-Open anchors: 400 paraphrases, 200 typed hard negatives, and 100 unrelated queries. Four matchers use stored nearest-neighbour scores, and threshold results are replayed from those records. At an exactly matched 40% cache-hit rate, cross-encoder reranking has the lowest construction-label false-hit rate (16.4%, 46/280), followed by BGE cosine (26.8%), MiniLM cosine (33.6%), and lexical similarity (36.4%). This reduction comes with a 41.5% false-miss rate on paraphrases for the cross-encoder. At τ=0.75, negation hard-negative hit rates are 98% for MiniLM and BGE cosine and 24% for cross-encoder reranking. These findings show a cost–correctness trade-off on this constructed workload; score cutoffs must be selected per matcher, and the observed ordering should not be assumed to transfer to other query populations.

Keywords: semantic caching, large language models, retrieval-augmented generation, similarity threshold, false-hit rate, hard negatives, LLM-as-judge

I. Introduction

Every call to a large language model costs compute and time, and for hosted models it costs money as well. Much of the traffic that reaches such systems is repetitive: people rephrase a question they or someone else has already asked, or send the same request in slightly different words. Exact-match caching misses nearly all of this, because a single changed word produces a different key. Semantic caching goes after the repetition directly. It encodes each incoming query as a vector, finds the closest previously answered query, and returns the stored response when the two are similar enough, so the model is never invoked. In a retrieval-augmented generation (RAG) pipeline [1] a hit also skips retrieval, because the cache sits in front of the whole pipeline. Open-source and research systems report the benefit in concrete terms: GPTCache reports responses that are 2 to 10 times faster when the cache is hit [2], and ContextCache reports cached responses with about ten times lower latency than a direct model call [3].

The difficulty is that a close vector is not the same thing as the same question. A query that swaps one city for another, flips a negation or changes a year can sit very near the original in embedding space while needing a different answer. When the cache returns the old answer anyway, we call it a false hit. A false hit is a worse outcome than a miss. A miss costs one extra model call, whereas a false hit delivers a wrong answer that looks like any other cached answer. Most caches control this risk with a single static similarity threshold. The vCache authors observe that static thresholds give no formal correctness guarantee and can produce unexpected error rates [4]. This is the problem we study, and we study it from a particular angle: not whether a better threshold policy exists, but what a given threshold actually tolerates and what it lets through.

The existing evidence on this question comes mostly from logged or benchmark traffic. MeanCache is evaluated against GPTCache on precision and F-score [5]. SCALM starts from an analysis of real chat data and reports gains in hit ratio and token savings [6]. Krites evaluates on trace-driven simulations built from two vCache benchmarks, one of Chatbot Arena prompts and one of search queries, and its main results approve a cached answer whenever both prompts share an equivalence class defined by the benchmark [7]. These are reasonable designs for their purposes. They share a limitation for our question, though. In logged traffic, nobody controls how many of the near-duplicates are true paraphrases and how many are look-alikes that need a different answer, and nobody can say which kind of look-alike caused a given error. An aggregate error rate hides that.

We take the opposite approach and construct the traffic. Each evaluation query is derived from a real question as a paraphrase, a hard negative with one of four named perturbation types, or an unrelated query, so its intended

relation to the cached question is controlled by construction. We replay stored nearest-neighbour scores across several matching methods and compare construction-label outcomes with the stored response-acceptability judgments. The paper makes four contributions.

1. A controlled benchmark for semantic caching in which every query is a construction-labelled paraphrase, a hard negative of a known type, or an unrelated query, built from real questions.
2. A validated measure hierarchy that uses construction-label error rates as primary outcomes when the response judge does not meet the prespecified acceptance and agreement gates, while retaining judge-based false-hit rate as a secondary diagnostic.
3. A like-for-like comparison of four matching configurations under one cost accounting: two bi-encoder models, bi-encoder retrieval with cross-encoder reranking, and lexical matching.
4. A replay design in which the nearest neighbour, the fallback model response and the judge verdict are computed once per query and reused across every threshold, so a full threshold sweep is cheap and exactly repeatable.

The rest of the paper is organized as follows. Section II reviews related work. Section III states the research gaps and our research questions. Section IV describes the methodology in full. Section V presents the analysis and discussion. Section VI covers threats to validity and Section VII concludes.

II. Background and Related Work

A. How a semantic cache works

A semantic cache holds pairs of a past query and the response that was generated for it. For an incoming query, an encoder maps the text to a vector. Sentence encoders such as Sentence-BERT [8] and SimCSE [9] are trained so that vectors of related sentences lie close together, and in the dense-retrieval setting the same idea is used to match questions against passages [10]. The cache looks up the cached query whose vector is closest, usually by cosine similarity, using exact search or an approximate index such as a graph-based one [11], [12]. If the similarity reaches a threshold τ, the cache returns the stored response and counts a hit. Otherwise it counts a miss and the model is called. We use *false hit* for a hit whose response is not acceptable for the incoming query, and *false miss* for a miss on a query whose paraphrase was in the cache. Both errors depend on the encoder, on the kind of query variation, and on where τ sits. Measuring that dependence directly is the aim of this paper.

B. Semantic caching systems

GPTCache is an open-source semantic cache for LLM applications. It stores model responses and returns them on a hit, and the paper reports 2 to 10 times faster responses on a hit together with response times that are not affected by network fluctuation [2]. Its reported results concern speed and cost, and the abstract does not describe a study of how correctness changes with the similarity threshold. GPT Semantic Cache builds a caching layer on embeddings and reports that it reduces API calls by up to 68.8% across query categories, with hit rates between 61.6% and 68.8% and positive hit rates above 97% [13]. SCALM starts from an analysis of real user-to-LLM chat data and argues that existing methods do not exploit semantic connections between queries. Its architecture identifies important cache entries and patterns, and it reports, relative to GPTCache, an average 63% higher cache hit ratio and 77% higher token savings [6].

Two works improve the similarity model itself. MeanCache runs on the user's side, trains its similarity model by federated learning so that user data stay private, and encodes context chains to tell contextual queries from standalone ones. Against GPTCache it reports about 17% higher F-score and 20% higher precision, an 83% smaller storage requirement, and 11% faster hit-or-miss decisions [5]. A report from Redis fine-tunes a compact encoder on Quora question pairs and on medical question pairs. On the medical data, precision rises from 78% to 92%, and a model trained on about 35,000 synthetic pairs alone reaches 87%, compared with 85% for OpenAI's text-embedding-3-large [14]. Both results show that the quality of the similarity function matters. Neither isolates what a fixed, off-the-shelf encoder does at a given threshold across different kinds of near-miss, which is what we measure.

A third group targets multi-turn conversation, where the same sentence can mean different things depending on what came before. ContextCache is a demonstration paper that adds context awareness to semantic caching for multi-turn queries [3]. SmartCache uses a semantic forest to index conversational turns, detects context changes from attention scores, and shares key-value cache state across sessions. On CoQA and SQuAD it reports up to 59.1% lower key-value memory use than prefix caching, 78.0% and 71.7% lower time-to-first-token, and higher

answer quality (39.9% higher F1 for one model on CoQA) [15]. Our setting is single-turn by design, so that the effect of paraphrase and perturbation is not mixed with the effect of dialogue history.

The closest work to ours treats correctness as the quantity to control. vCache describes itself as the first verified semantic cache with user-defined error-rate guarantees. It replaces the single static threshold with a policy that meets a bound the user sets, and it reports that it consistently meets the specified bounds while outperforming static-threshold and fine-tuned embedding baselines [4]. Krites keeps the serving path of an ordinary static-threshold cache and adds an asynchronous LLM judge. When a prompt's nearest curated answer falls just below the threshold, the judge decides off the critical path whether the answer fits, and an approved answer is promoted into the dynamic cache. On trace-driven simulations it raises the fraction of requests served from curated answers from 8.2% to 19.4% on conversational prompts and from 2.2% to 8.6% on search queries, with critical-path latency unchanged. Its main results use an oracle judge based on the benchmark's equivalence classes, and a separate check found that a real LLM judge agreed with task labels on 99 of 100 borderline pairs [7]. Both papers show that the threshold–correctness relationship matters in practice. What they do not report, in the material we reviewed, is a sweep over a corpus where the difficulty of the near-misses is itself set and labeled by type.

C. Neighbouring work on LLM cost

Some cost-saving work sits next to response caching but is a different thing. FrugalGPT proposes strategies to cut LLM cost, including prompt adaptation, LLM approximation and an LLM cascade that learns which model to use for each query. It reports matching the best single model (GPT-4) with up to 98% lower cost, or improving accuracy over GPT-4 by 4% at the same cost [16]. RAGCache stores the intermediate key-value states of retrieved documents in a GPU and host-memory hierarchy and reports up to 4 times lower time-to-first-token and up to 2.1 times higher throughput than vLLM with Faiss [17]. Prompt Cache reuses attention states for recurring prompt segments and reports time-to-first-token speedups of about 8 times on GPU and 60 times on CPU [18]. All three save work inside or around generation. A response-level semantic cache, our subject, decides whether generation can be skipped altogether, and so it alone carries the risk of returning an answer to the wrong question.

D. Perturbation-based evaluation in NLP

The idea of testing a model with small, targeted edits is established in NLP, and we borrow its logic. PAWS builds 108,463 paraphrase and non-paraphrase pairs with high lexical overlap, and it shows that models trained on earlier paraphrase datasets perform very poorly on it [19]. Contrast sets ask dataset authors to perturb test instances in small ways that usually change the gold label, and for ten datasets they report performance up to 25% lower than on the original test sets [20]. CheckList organizes behavioural tests as a matrix of linguistic capabilities and test types, and it found critical failures in commercial and state-of-the-art models [21]. For negation in particular, an analysis of large language models finds insensitivity to the presence of negation, an inability to capture its lexical semantics, and a failure to reason under negation [22]. That last finding is a reason to expect negation to be a hard case for similarity-based matching, though we treat it as something to measure and not to assume. None of these resources is organized as a cache. They have no cached responses, no threshold, and no hit or miss, which is why we build a corpus of our own. Table II places them next to our design.

E. LLM judges

Using a language model to grade outputs is now common, and its limitations are documented. MT-Bench reports over 80% pairwise preference agreement for strong judges such as GPT-4 while examining position, verbosity and self-enhancement biases [23]. G-Eval reports a Spearman correlation of 0.514 on summarization judgments and notes a possible preference for model-generated text [24]. Later work links self-preference to a model’s recognition of its own outputs [25]. Krites reports agreement between its language-model judge and benchmark labels on 99 of 100 borderline pairs [7]. These studies use different tasks and measures from this paper’s cached-response evaluation; Table III makes the distinctions explicit.

TABLE I. Semantic-cache and related LLM-cost work compared with this paper.

Work	Contribution	Reported evidence	Relation to this paper
GPTCache [2]	Open-source cache that stores LLM responses and serves them on a hit	2–10× faster responses on a hit	We study correctness as a function of threshold, which the abstract does not
GPT Semantic Cache [13]	Embedding-based caching layer	Up to 68.8% fewer API calls; hit rates 61.6–68.8%; positive hit rate above 97%	Same goal; we add typed near-misses and a judge on the reused response
SCALM [6]	Cache built on analysis of real chat data	Relative to GPTCache, on average: +63% hit ratio, +77% token savings	Built from an analysis of real chat data; we control the query mix
MeanCache [5]	User-side cache, federated similarity model, context chains	~17% higher F-score, ~20% higher precision, 83% less storage vs. GPTCache	They improve the similarity model; we measure fixed encoders
Redis domain embeddings [14]	Fine-tuned compact encoder with synthetic data	Medical precision 78% → 92%; synthetic-only 87%	Complementary; we do not fine-tune
ContextCache [3]	Context-aware caching for multi-turn queries	About 10× lower latency for cached responses	Multi-turn; ours is single-turn
SmartCache [15]	Semantic forest, context-change detection, cross-session KV sharing	Up to 59.1% less KV memory; TTFT −78.0% and −71.7%	Multi-turn and KV-level; ours is response-level
vCache [4]	Verified cache with user-defined error-rate bound	Meets specified bounds; beats static and fine-tuned baselines	Closest in aim; we characterize what a static threshold tolerates by type
Krites [7]	Asynchronous LLM-judge promotion of curated answers	Static-origin served fraction 8.2→19.4% and 2.2→8.6%	Judge inside the policy; ours is offline evaluation on a controlled corpus
FrugalGPT [16]	LLM cascade and related cost strategies	Up to 98% cost reduction at GPT-4 quality	Model selection, not response reuse
RAGCache [17]	KV-state cache for retrieved documents	TTFT up to 4×, throughput up to 2.1×	Below the response layer
Prompt Cache [18]	Attention-state reuse for prompt segments	TTFT ~8× (GPU), ~60× (CPU)	Below the response layer

TABLE II. Perturbation-based benchmarks compared with this paper's corpus.

Resource	How the variants are made	Organized by named perturbation type	Cache entries and a threshold
PAWS [19]	Word swapping and back translation; high lexical overlap	No	No
Contrast sets [20]	Dataset authors perturb test instances so the label usually changes	By dataset	No
CheckList [21]	Matrix of linguistic capabilities and test types	Yes, by capability and test type	No
This paper	LLM-generated paraphrases at four levels; hard negatives of four types; reviewed on a 90-item stratified audit	Yes: entity, negation, numeric or temporal, scope	Yes

TABLE III. Prior findings on judge agreement, error control and negation.

Prior work	Reported result or method	Relation to this study	Caution in comparison
MT-Bench [23]	Strong judges including GPT-4 exceed 80% pairwise preference agreement.	Phi-4-mini has 71.1% raw agreement and κ=0.416 against this audit consensus.	Tasks, labels, models and samples differ.

Prior work	Reported result or method	Relation to this study	Caution in comparison
Krites [7]	A separate LLM judge agreed with benchmark labels on 99 of 100 borderline pairs.	This study finds substantial judge false rejection and false acceptance on cached-response pairs.	Krites uses a different benchmark and judge protocol.
vCache [4]	A verified policy targets a user-defined error bound.	The cross-encoder's JFH stays between 35.8% and 37.8% across τ=0.50–0.90, but failed judge validation means this flat judge error is not evidence of flat cache error.	Our judge is not validated; unlike vCache, JFH is not a verified error bound.
Truong et al. [22]	Reports language-model difficulties with negation.	At τ=0.75, MiniLM and BGE hit 49/50 negation queries; cross-encoder hits 12/50.	Our result is cache retrieval on generated queries, not QA reasoning.

III. Research Gaps

Reading across these works, we see four gaps. Each one leads to a research question that the rest of the paper answers.

Gap 1: no typed account of what a threshold lets through. The caching papers above report aggregate quantities such as hit rate, precision, F-score or an error bound. In the material we reviewed, none reports how the false-hit rate changes with the threshold separately for entity substitution, negation, numeric or temporal change and scope change. *RQ1: How does the false-hit rate move with the threshold for each type of near-miss, and which types does a given threshold fail on first?*

Gap 2: correctness is mostly defined at the level of query labels. Krites's main results approve a cached answer when two prompts share a benchmark class [7], and a pair can share a class while the particular stored response still fails the new query. *RQ2: How often does a judge's verdict on the reused response disagree with the label of the matched pair, and for which query types?*

Gap 3: perturbation testing exists, but not for caches. PAWS, contrast sets and CheckList show how much a targeted edit can reveal, yet none has cache entries, stored responses or a threshold. *RQ3: Can a typed corpus built from real questions expose differences between matching methods that aggregate evaluation hides?*

Gap 4: matching methods are rarely compared under one corpus and one cost accounting. The caching papers evaluate their own matcher against a baseline. *RQ4: Under identical queries, an identical cache and an identical judge, how do bi-encoder matching with two embedding models, cross-encoder reranking and lexical matching trade false hits against cost savings?*

IV. Methodology

A. Overview

The methodology has two halves. The first half builds a corpus in which the relationship between every evaluation query and the cache is known. The second half runs matching methods over that corpus, scores every hit, and sweeps the threshold. Fig. 1 shows the whole flow. The design rule that holds it together is that anything expensive, such as a model call, is performed once per query and stored. Thresholds are applied afterwards by replaying stored results.

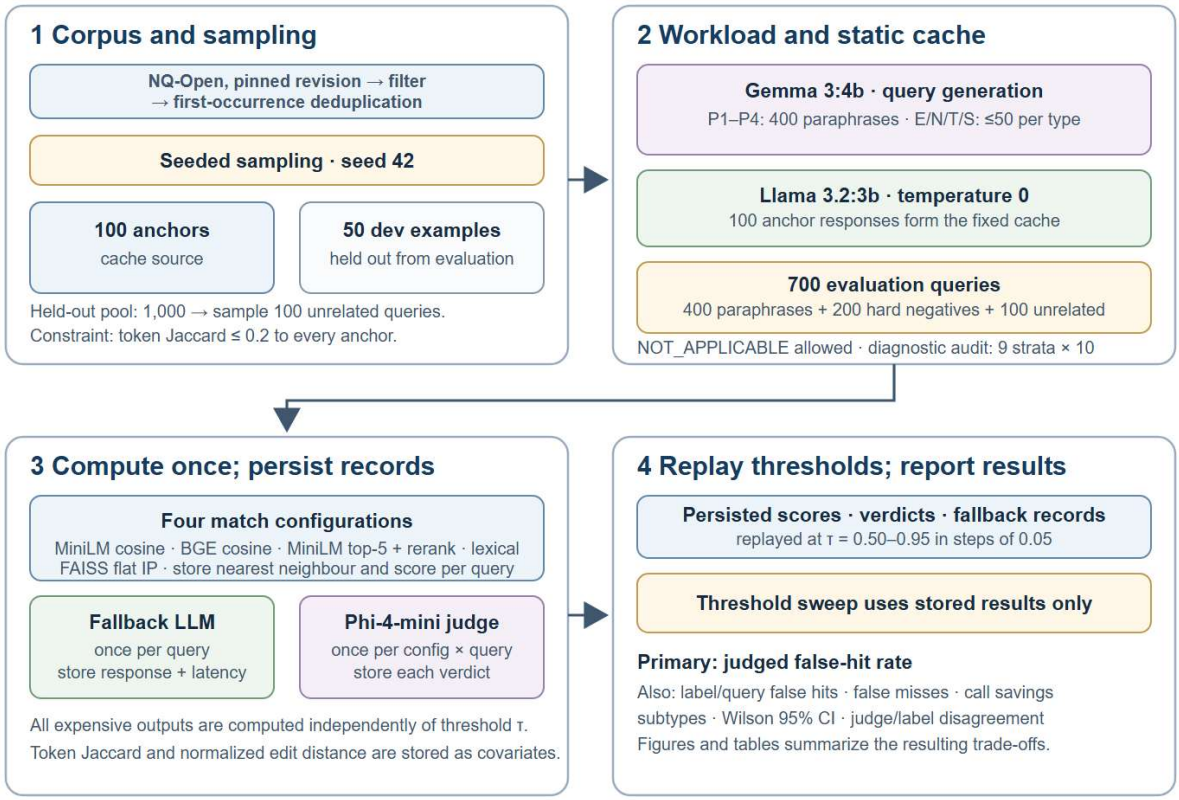


Fig. 1. End-to-end evaluation pipeline. The development split is excluded from evaluation. Retrieval outputs, fallback responses and latency, and judge verdicts are persisted once, then reused across the threshold grid.

B. Problem formulation and metrics

Setting. The cache is a fixed set of anchor questions C, each stored with a response r(a) generated by the fallback model. The cache is static: a miss does not insert the new query. We chose this so that the outcome for one query never depends on the order of other queries, which is what makes replay across thresholds exact. Eviction is therefore out of scope, and Section VI discusses what that leaves out.

For an incoming query q, a matching configuration returns a candidate anchor a*(q) in C and a score s(q). The cache declares a hit at threshold τ when s(q) ≥ τ and a miss otherwise (1). On a hit, the response r(a*(q)) is served. On a miss, the fallback model is called.

$$\text{hit}_\tau(q) = 1 \text{ if } s(q) \geq \tau, \text{ else } 0 \tag{1}$$

For the bi-encoder configurations, s(q) is the largest cosine similarity between the vector of q and the vector of any anchor, and a*(q) is the anchor that achieves it. The other configurations define s(q) in Section IV-E.

Secondary measure: judged false-hit rate (JFH). For every query and matcher, Phi-4-mini receives the incoming query, the matched anchor and the stored response, and returns an acceptability verdict. JFH is the share of cache hits the judge marks unacceptable (2). The validation results in Section V-F show that this judge does not pass the prespecified acceptance and agreement gates; JFH is therefore a secondary diagnostic, not the primary correctness estimate.

$$\text{JFH}(\tau) = |\{q : \text{hit}_\tau(q) \text{ and } J(q) = \text{unacceptable}\}| \,/\, |\{q : \text{hit}_\tau(q)\}| \tag{2}$$

Primary construction-label measures. Each cache hit is evaluated against its construction label and retrieved anchor. A true hit (TH) is a paraphrase whose nearest neighbour is its own source anchor. Every other cache hit is a label false hit (FH), including hard negatives, unrelated queries and paraphrases retrieved from the wrong anchor. The label false-hit rate (FHR) is conditional on hits (3). Query-level false-hit incidence is FH divided by all evaluation queries, allowing comparison at equal traffic volume. These outcomes measure agreement with the constructed query–anchor relation; they do not independently establish whether a stored response is factually correct for a particular query.

$$\text{FHR}(\tau) = \text{FH}(\tau) \,/\, (\text{TH}(\tau) + \text{FH}(\tau)) \tag{3}$$

$$\text{QFHI}(\tau) = \text{FH}(\tau) \,/\, |Q|$$

If JFH and FHR diverge, that divergence is a result. It means the judge accepts responses to label-mismatched pairs, or rejects responses to correctly matched ones. We report judge-versus-label disagreement among hits, including by query type.

False misses. Let Q_para be the paraphrase queries. Each has its anchor in the cache, so a miss on a paraphrase query is a false miss (4).

$$\mathrm{FMR}(\tau) = |\{q \text{ in } Q_para : \mathrm{hit}_\tau(q) = 0\}| \,/\, |Q_para| \tag{4}$$

Cost. Each miss triggers exactly one fallback call, so model-call savings are the share of queries that hit cache (5). For descriptive context we also report the mean stored fallback latency for queries served from cache. This is the fallback latency avoided before accounting for cache lookup time; it is not a net end-to-end latency saving.

$$\mathrm{CS}(\tau) = 1 - |\{q : \mathrm{hit}_\tau(q) = 0\}| \,/\, |Q| \tag{5}$$

Cost is therefore expressed as model calls avoided and stored fallback latency avoided, not as API prices or net serving latency.

Uncertainty. Every rate is a proportion k out of n and is reported with a Wilson 95% interval (6), where $\hat{p} = k/n$ and $z = 1.96$. We use the Wilson interval because it behaves well for small samples and for proportions near 0 or 1, where the simple normal approximation does not [26], [27].

$$[\, \hat{p} + z^2/(2n) \pm z \cdot \mathrm{sqrt}(\, \hat{p}(1-\hat{p})/n + z^2/(4n^2)\,)\,] \,/\, (\,1 + z^2/n\,) \tag{6}$$

The primary cost–correctness curve plots construction-label FHR against model-call savings as the threshold varies. JFH is shown separately as a judge-based diagnostic. A threshold with no hits has zero conditional FHR but provides no cache service.

C. Corpus construction

Anchor source. Anchors come from NQ-Open, the open-domain version of Natural Questions introduced for open-domain question answering [28]. Natural Questions consists of real, anonymized queries issued to Google Search [29]. We chose real questions so that the surface forms of the anchors are not themselves a product of our generator. We load the dataset at a pinned revision so that the same questions are drawn on every run.

Filtering and deduplication. A mechanical filter removes records with an empty question, control characters, no alphabetic character, a URL or markup, a token count outside a fixed range, or one token that dominates the question, and records with no usable answer. Tokens are lowercase runs of Unicode letters and digits, so punctuation and underscores separate tokens. Near-duplicate questions are then removed using the Jaccard similarity of token sets, with the first occurrence kept. The comparison uses a prefix filter, which indexes only the rarest tokens of each set so that most pairs are never compared and every pair that could reach the threshold is still checked exactly [30]. Deduplication matters here for a concrete reason: duplicated text is common in datasets and inflates results when the same content appears on both sides of a comparison [31]. In our case a near-duplicate of an anchor sitting among the unrelated queries would be a labeling error.

Sampling. With a fixed seed we draw 100 anchors, 50 separate development anchors, and a held-out pool of 1,000 questions. The development anchors are used only to debug prompts and the pipeline, and they never enter the cache or the reported results. The 100 unrelated queries come from the held-out pool and must have a token-level Jaccard similarity of at most 0.2 with every anchor.

Paraphrases. For each anchor the generator writes four paraphrases, one for each of four intended difficulty levels that run from light word substitution (P1) to heavy rewording (P4). For every paraphrase we record two covariates: its token-level Jaccard overlap with the anchor and its normalized edit distance from the anchor. These measured values let us check that the intended levels are ordered the way we intended, and they let us analyze false misses against measured distance and not only against the label.

Hard negatives. The hard negatives are the part of the corpus that does the real testing. A cache that only rejects dissimilar text is not being tested at all. The queries that expose a cache are those that look alike and need a different answer. We generate them along four types, each defined relative to the anchor and abbreviated by its initial.

TABLE IV. Hard-negative types, with illustrative pairs written for this paper (not drawn from the corpus).

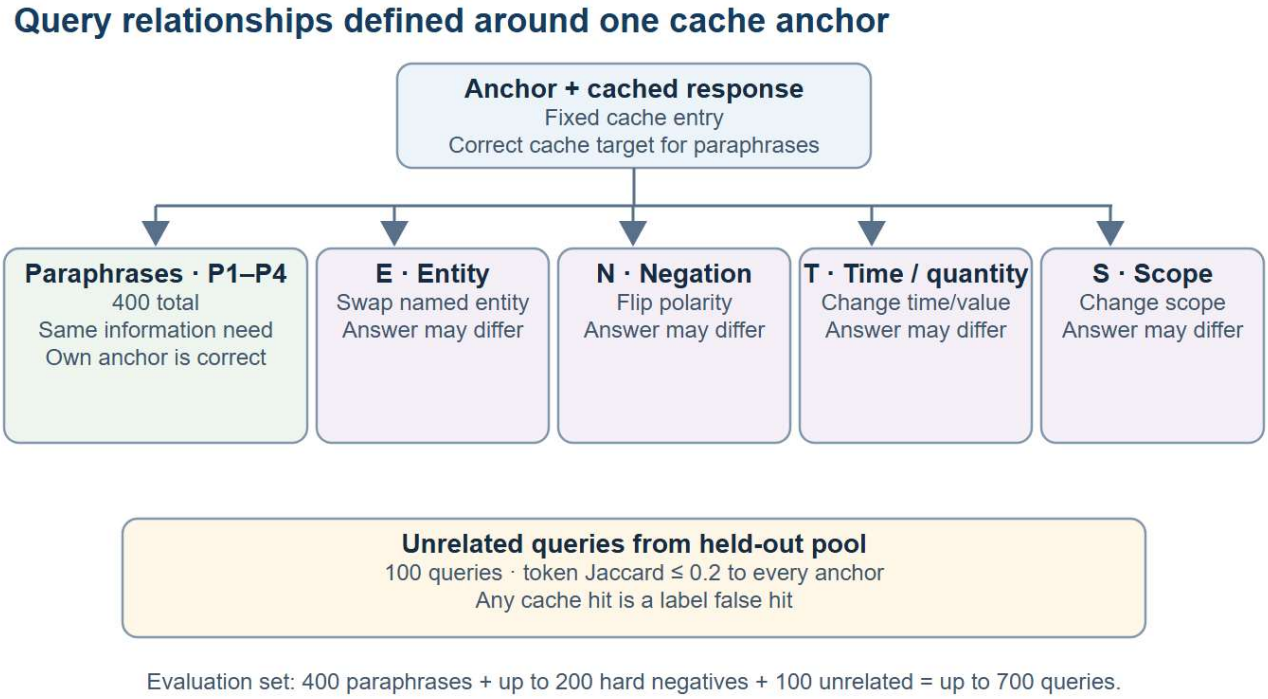
Type	Definition	Anchor	Hard negative
E, entity substitution	Same question shape, a different named entity	who directed the film Inception	who directed the film Interstellar
N, negation or polarity flip	Same shape, opposite truth condition	which planets have rings	which planets do not have rings
T, numeric or temporal change	Same shape, a different quantity, date or range	who won the FIFA World Cup in 2018	who won the FIFA World Cup in 2014
S, scope change	Same shape, a narrower or broader subject	what is the population of Jaipur	what is the population of Rajasthan

The generator produces up to 50 hard negatives for each type. When a perturbation does not apply to an anchor, the generator may answer that it does not apply, and we skip the anchor. We never force a perturbation, because a forced hard negative is likely to be a bad one. A generated query is rejected if it is not valid output, if it equals the anchor after normalization or equals any paraphrase of the same anchor, or if its length differs from the anchor's by more than a factor of two. Each rejected query is regenerated up to three times. Each type is tagged on the individual query, so that every result can be broken down by type.

Unrelated queries and cached responses. The unrelated queries serve as a floor check and not as a discriminating test: any hit on one is a false hit. For each anchor, the fallback model writes a response once, at temperature 0, and that response is stored in the cache.

Evaluation queries. The evaluation set therefore contains 400 paraphrases, at most 200 hard negatives and 100 unrelated queries, so at most 700 queries.

Fig. 2. Structure around one anchor.



D. Label verification

We assess the constructed labels with a stratified 90-item audit: ten items from each of P1–P4, E/N/T/S, and the unrelated stratum. The items were labelled by several independent LLM raters—ChatGPT GPT-6 Luna, Sonnet 5.5, and Claude Sonnet 4.6 (Thinking)—who received the query, matched anchor and cached response without sight of the intended label, and supplied yes/no ratings for information-need match, answer difference, and cached-response acceptability. The saved majority labels are compared with Phi-4-mini. The final evaluation set contains 700 queries after 37 recovered paraphrase records were incorporated. We prespecified two gates for making JFH primary: at least 90% acceptance on the 100 anchor self-pairs and Cohen's $\kappa \geq 0.60$ against the audit consensus. Cohen's κ corrects observed agreement for chance agreement [39]; its interpretation is treated cautiously using the conventional categories of Landis and Koch [40]. If either gate fails, construction-label FHR and hard-negative hit rate are primary, with JFH secondary and the judge's false-rejection rate reported. The anchor self-pair rate reflects both the stored response quality and the judge decision, so it is a pipeline gate rather than a stand-alone estimate of judge accuracy.

E. Matching configurations

We compare four configurations. They differ in how they compute the candidate a*(q) and the score s(q), and they share everything else.

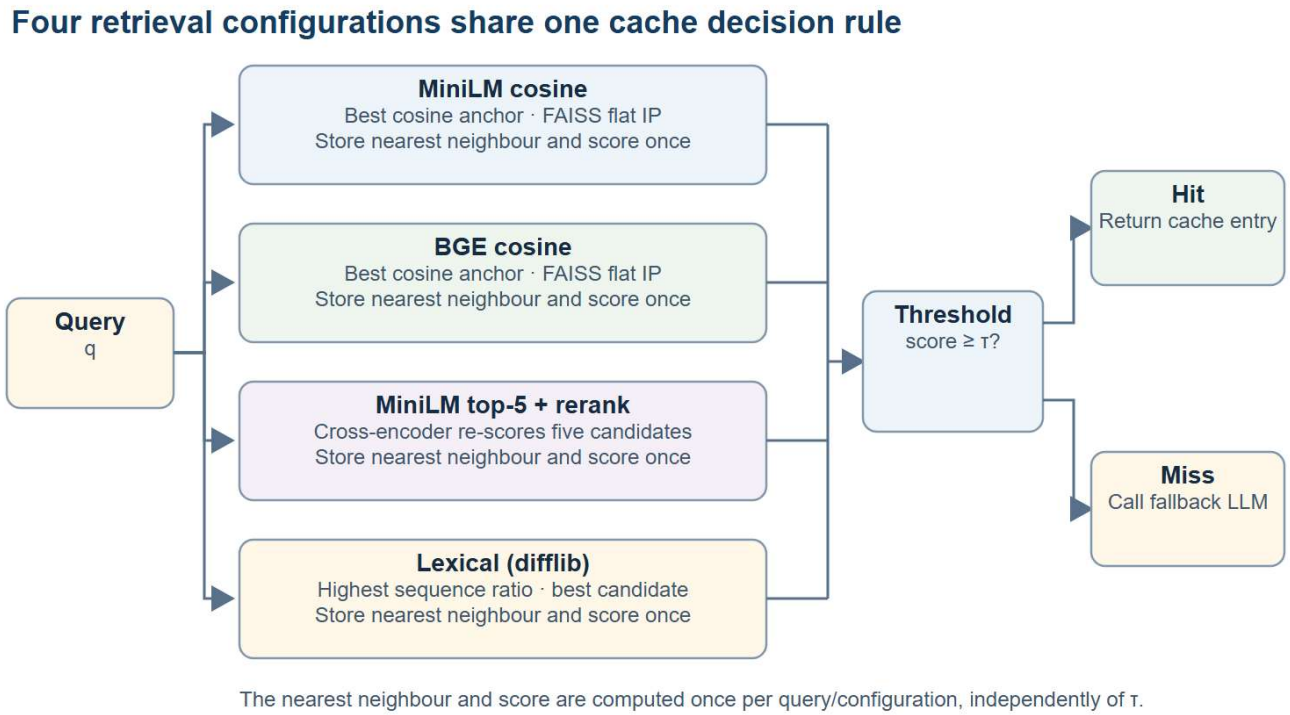
TABLE V. Matching configurations.

Configuration	Candidate and score
Bi-encoder with all-MiniLM-L6-v2	Largest cosine similarity over all anchors; vectors are 384-dimensional
Bi-encoder with bge-base-en-v1.5	Largest cosine similarity over all anchors; vectors are 768-dimensional
Bi-encoder with cross-encoder reranking	The five closest anchors under all-MiniLM-L6-v2 are re-scored by a cross-encoder; s(q) is the highest score and a*(q) its anchor
Lexical	The ratio of the Python difflib sequence matcher between lowercased texts, with the highest-ratio anchor as candidate

The first two use a FAISS flat index over L2-normalised vectors, for which the inner product equals the cosine similarity [11]. The flat index performs exact search, so no result depends on an approximation. all-MiniLM-L6-v2 is a sentence encoder built on the MiniLM architecture [32], used through the Sentence-BERT framework [8].

bge-base-en-v1.5 belongs to the BGE family of embedding models [33], whose authors report state-of-the-art results for the English models on the MTEB benchmark at release [33], [34]. The third configuration follows the retrieve-then-rerank pattern, in which a cross-encoder that reads the query and the candidate together re-scores candidates found by a cheaper retriever [35]. The cross-encoder is a model trained on duplicate-question pairs and returns a score between 0 and 1. The fourth configuration is deliberately simple. It has no learned component and shows what surface similarity alone achieves.

Fig. 3. The four configurations.



F. Judge protocol

The judge is Phi-4-mini, run locally [36]. It differs in model family from Gemma 3, which generated the corpus [37], and Llama 3.2 3B, which generated cached responses [38]. We use the unchanged prompt printed in Appendix A at temperature 0. The same prompt was applied once to each of the 100 anchor questions paired with its own stored response; all verdicts and prompt hashes are persisted. No prompt revision was made after examining results.

G. Experimental setup

All experiments run on one consumer machine with an AMD Ryzen 5 processor and integrated graphics. There is no discrete GPU, so every model runs on the CPU. The small size of the models and the corpus follows from this constraint. Latencies reported in this paper describe CPU inference on this class of hardware, not a GPU server.

TABLE VI. Components and settings.

Component	Choice
Anchor source	NQ-Open [28], pinned revision
Corpus generator	Gemma 3, 4B parameters, through Ollama [37]
Fallback model and stored responses	Llama 3.2, 3B parameters, through Ollama, temperature 0 [38]
Judge	Phi-4-mini, 3.8B parameters, through Ollama, temperature 0 [36]
Bi-encoders	all-MiniLM-L6-v2 and bge-base-en-v1.5 [8], [33]
Cross-encoder	cross-encoder/quora-distilroberta-base
Vector index	FAISS flat inner-product index over normalised vectors [11]
Lexical matcher	Python difflib sequence ratio
Threshold grid	0.50 to 0.95 in steps of 0.05 (ten values)
Seed	42 for all sampling

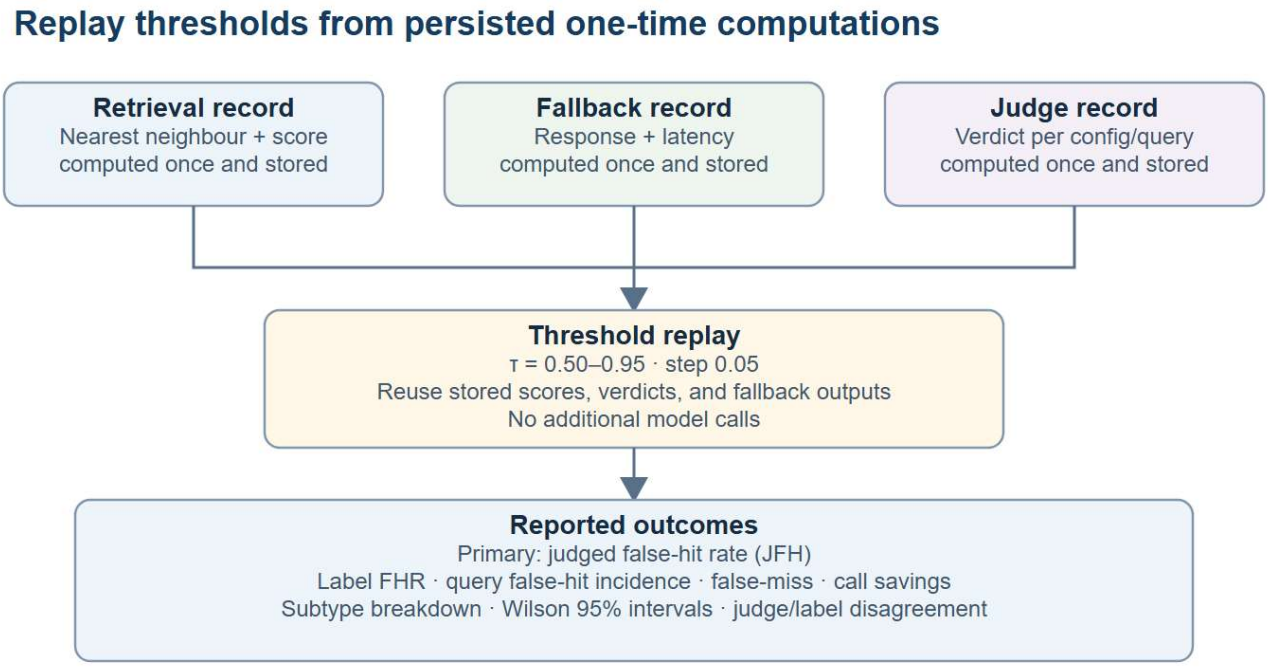
H. Experiment protocol

Stage 1: build and verify. The corpus is generated, filtered and deduplicated as described in Section IV-C; the cache contains the 100 anchors and their stored responses.

Stage 2: compute once. For every query and configuration, compute the candidate and score once. Call the fallback model once per query and store its response, latency and output length. Call Phi-4-mini once per configuration–query pair and persist the verdict, independent of threshold.

Stage 3: replay. Apply thresholds to stored scores. For cross-matcher comparison, rank the 700 stored scores within each configuration and retain exactly the top 280 queries, giving an equal 40% cache-hit share; this score-only replay is independent of observed error outcomes. Report the corresponding configuration-specific score cutoffs. No model calls are made during threshold or matched-savings replay.

Fig. 4. Replay design.



The primary analysis reports per-configuration threshold curves and an exact matched-savings comparison based on the top 280 stored scores per configuration. Because score scales differ, results at the same numeric τ are not treated as a fair matcher comparison.

Breakdown. All rates are computed overall and by query type: the four paraphrase levels, the four hard-negative types, and the unrelated set. The gap between JFH and FHR is reported by type.

Matched-savings comparison. The 40% hit-share point is derived by ranking stored scores within each matcher and retaining 280 of the same 700 queries. It is an in-sample descriptive comparison, not a deployment-selected operating point.

Uncertainty. The corpus is generated once with a fixed seed and every model call uses temperature 0, so the experiment is one deterministic run, and no variance across seeds is estimated. The Wilson intervals treat queries as independent, although the paraphrases and hard negatives of one anchor are not. The intervals are therefore somewhat narrower than the true uncertainty, and small differences between configurations should not be read as established.

I. Reproducibility

Code, configuration and generated data are published at github.com/garvjain7/semantic-caching-research-paper (branch develop). One configuration file fixes the dataset revision, the seed, the corpus sizes, the model identifiers and the threshold grid, and one entry point rebuilds the corpus from it. The release includes the evaluation corpus with its labels and covariates, the filtered anchor list, the stored responses and latencies, the judge verdicts, the audit sheet and a manifest of the model builds used. Because the sweep replays stored results, every figure and table can be regenerated without running a model. Dataset sampling, filtering, deduplication, the sweep and all metrics are deterministic given the configuration. The generated paraphrases, hard negatives and responses depend on the exact model builds, so we release them as files and do not expect them to regenerate identically.

V. Results and Discussion

The results below use the saved 700-query evaluation set and persisted retrieval, fallback and judge records. Threshold curves replay stored nearest-neighbour scores. Construction-label FHR and hard-negative hit rate are primary because the judge fails both validation gates; JFH remains a secondary diagnostic.

A. Evaluation set

The evaluation set contains 400 paraphrases (100 per level), 200 hard negatives (50 per type), and 100 unrelated queries. The 100 cache anchors are fixed across configurations. Token Jaccard and normalized edit distance are measured against the source anchor for paraphrases and hard negatives, and against the maximum-overlap anchor

for unrelated queries. The strata are not ordered uniformly by either lexical measure; negation queries have high token overlap.

TABLE VII. Evaluation-set composition and mean lexical covariates relative to the source or maximum-overlap anchor.

Stratum	n	Mean token Jaccard	Mean normalized edit distance
P1	100	0.4175	0.4472
P2	100	0.3517	0.5549
P3	100	0.3432	0.5600
P4	100	0.2814	0.6211
Entity (E)	50	0.3102	0.5714
Negation (N)	50	0.7371	0.1852
Numeric/temporal (T)	50	0.3079	0.5870
Scope (S)	50	0.2791	0.6181
Unrelated	100	0.1543	0.6957

B. Overall threshold behaviour (RQ1, RQ4)

Figures 5 and 6 show within-configuration threshold sweeps for secondary JFH and model-call savings. A shared numeric τ is not comparable across cosine, cross-encoder probability, and lexical-ratio scales. Figure 7 and Table VIII instead compare matchers at exactly 40% cache calls using their stored-score rankings. The cross-encoder yields the lowest construction-label FHR at this service level, while JFH stays higher because the judge rejects many true paraphrase hits.

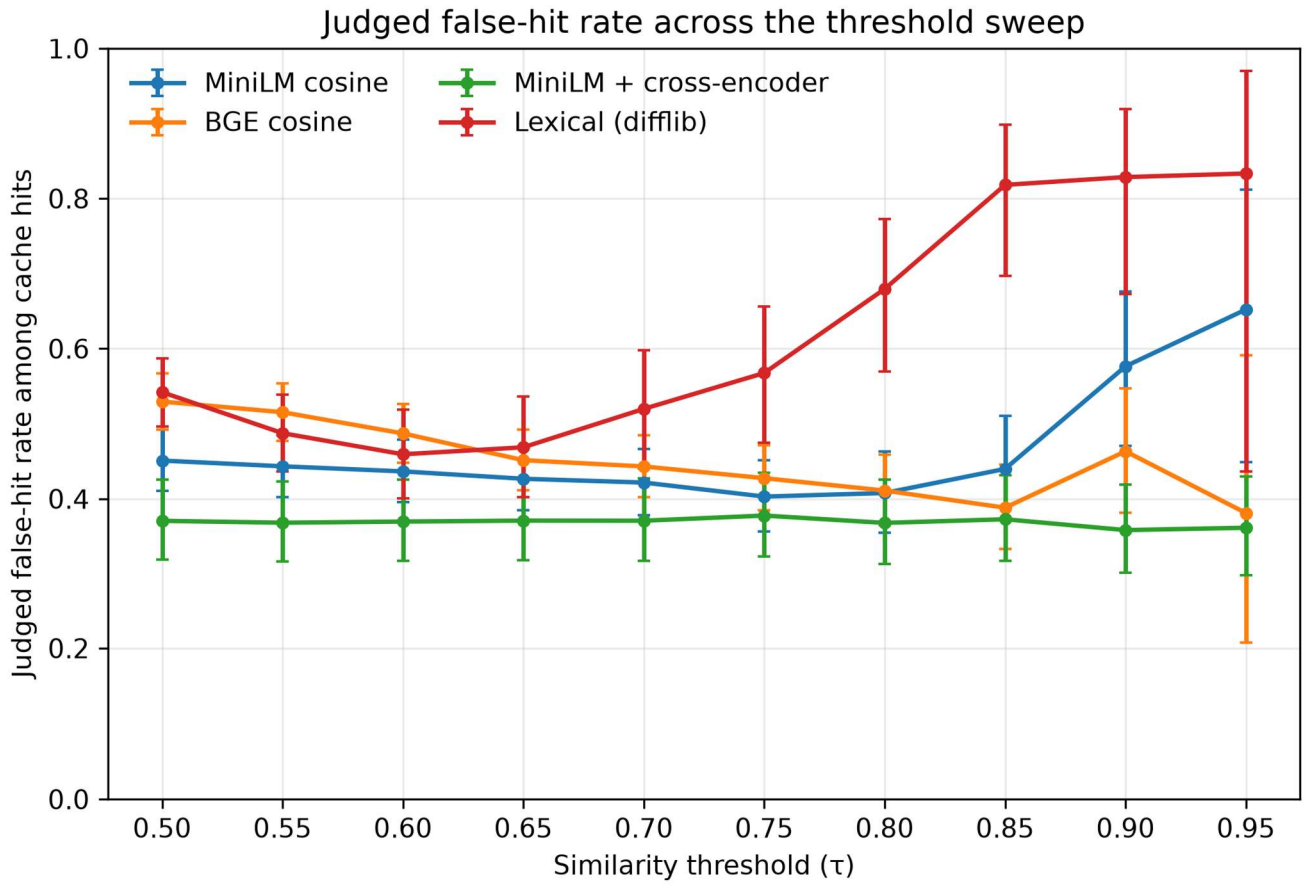


FIG. 5. Secondary judge-based false-hit rate over the threshold grid; the judge fails the prespecified acceptance and audit-agreement gates.

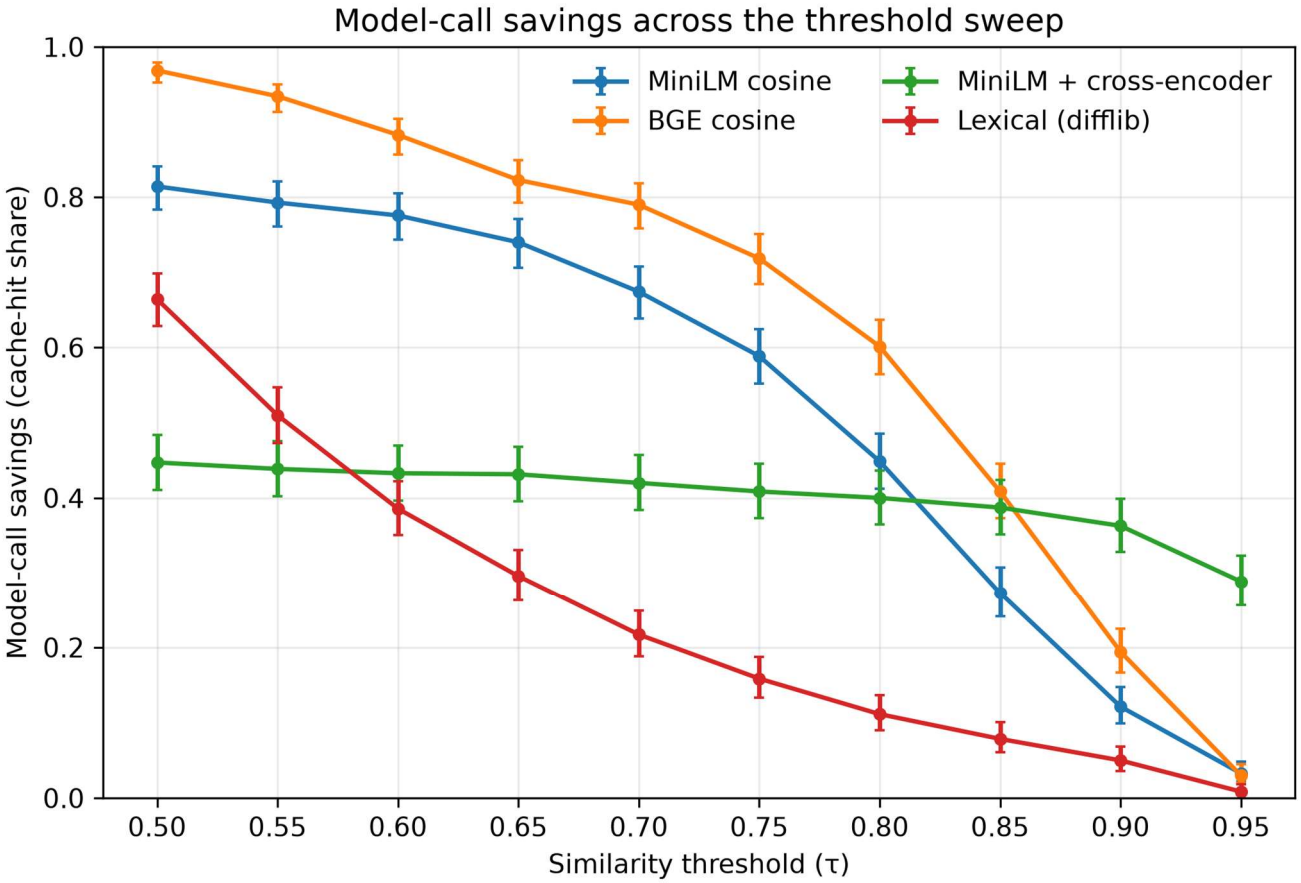


FIG. 6. Model-call savings over each configuration's threshold grid; numeric thresholds are not compared across score scales.

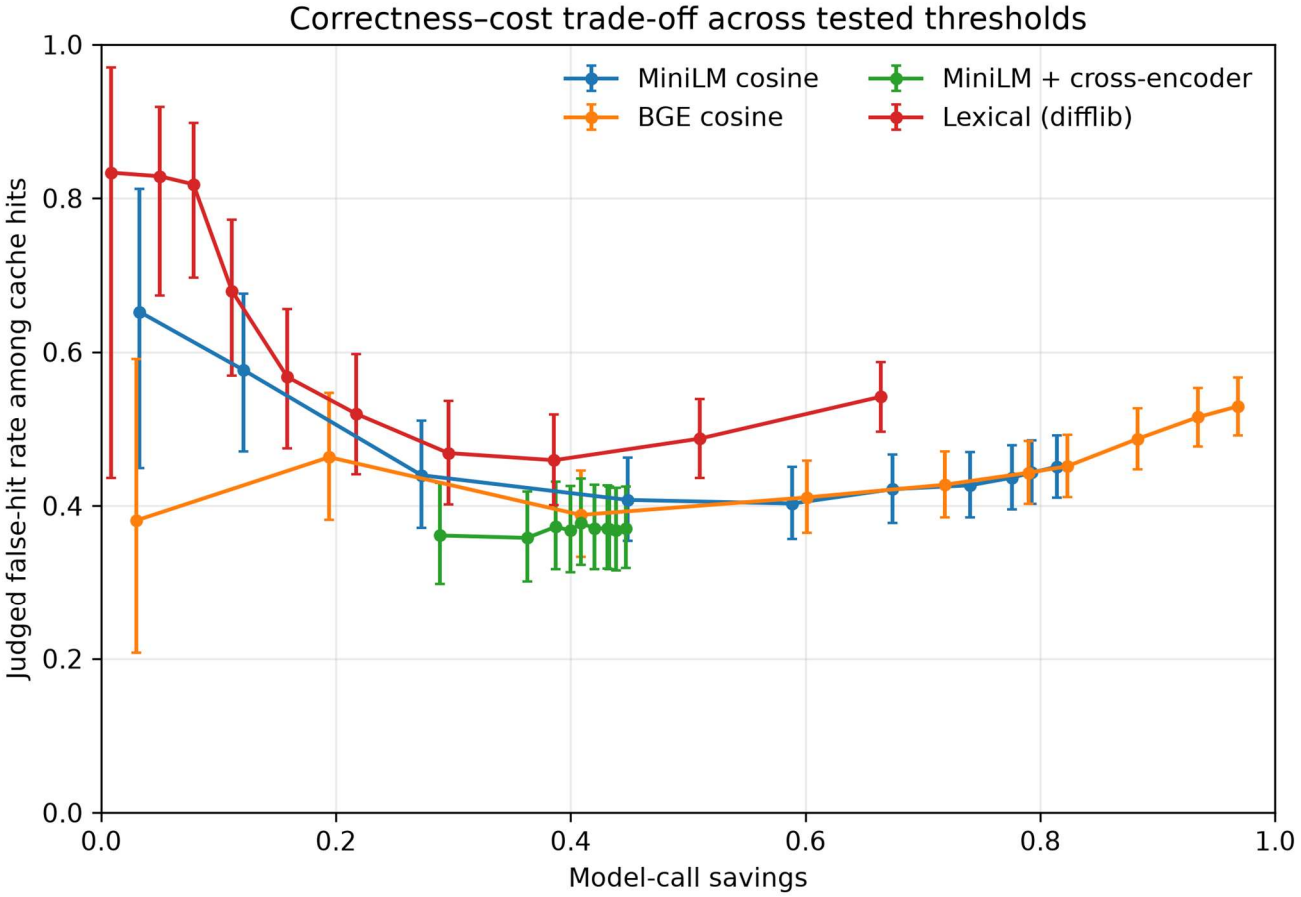


FIG. 7. Construction-label cost–correctness frontier over stored threshold results. Diamonds mark the exact 40% hit-share replay; bars show 95% Wilson intervals for label FHR.

At the matched 40% cache-hit share, cross-encoder reranking has the lowest construction-label FHR, while BGE and MiniLM occupy intermediate points and lexical matching has the highest FHR. The cross-encoder reduces hard-negative hits but also misses more paraphrases; Table VIII reports the exact rates and uncertainty intervals for all four matchers.

C. Matched-savings comparison

Table VIII reports the exact matched-savings replay. At 40% cache calls, construction-label FHR ranges from 16.4% for cross-encoder reranking to 36.4% for lexical matching. The cross-encoder also misses 41.5% of paraphrase queries at this operating point. Wilson intervals overlap for some matcher pairs; they are descriptive intervals, not paired significance tests, and query dependence may make them too narrow.

TABLE VIII. Matcher comparison at exactly 40% model-call savings.

MiniLM cosine	0.8163	280/700 (40.0%)	94/280 (33.6%; 28.3%–39.3%)	94/700 (13.4%)	214/400 (53.5%)	116/280 (41.4%; 35.8%–47.3%)
BGE cosine	0.8526	280/700 (40.0%)	75/280 (26.8%; 21.9%–32.3%)	75/700 (10.7%)	195/400 (48.8%)	108/280 (38.6%; 33.1%–44.4%)
MiniLM + cross-encoder	0.8009	280/700 (40.0%)	46/280 (16.4%; 12.5%–21.2%)	46/700 (6.6%)	166/400 (41.5%)	103/280 (36.8%; 31.4%–42.6%)
Lexical (difflib)	0.5941	280/700 (40.0%)	102/280 (36.4%; 31.0%–42.2%)	102/700 (14.6%)	220/400 (55.0%)	130/280 (46.4%; 40.7%–52.3%)

D. Hard-negative subtypes (RQ1, RQ3)

At τ=0.75, negation hit rates are 49/50 (98%) for MiniLM cosine and BGE cosine, 46/50 (92%) for lexical matching, and 12/50 (24%) for cross-encoder reranking. These are within-matcher results on the shared grid, not cross-matcher comparisons at a common operating point. Table IX reports hard-negative hit rates at equal 40% cache calls and JFH by subtype as a secondary diagnostic.

The pattern is consistent with the measured workload and matcher objectives. Negation queries average token Jaccard 0.737, so they remain close to their anchors despite reversing polarity. The selected cross-encoder, cross-encoder/quora-distilroberta-base, was trained for duplicate-question detection rather than cached-response acceptability [41]; it sharply lowers negation hits, but its paraphrase false-miss rate is 40.5% at τ=0.75 and 41.5% at the matched 40% point. The generator's level labels do not form lexical-distance bands: mean Jaccard is 0.4175 for P1, 0.3517 for P2 and 0.3432 for P3. P2 and P3 are nearly indistinguishable by this measure, so the results are interpreted by construction subtype and measured covariates separately.

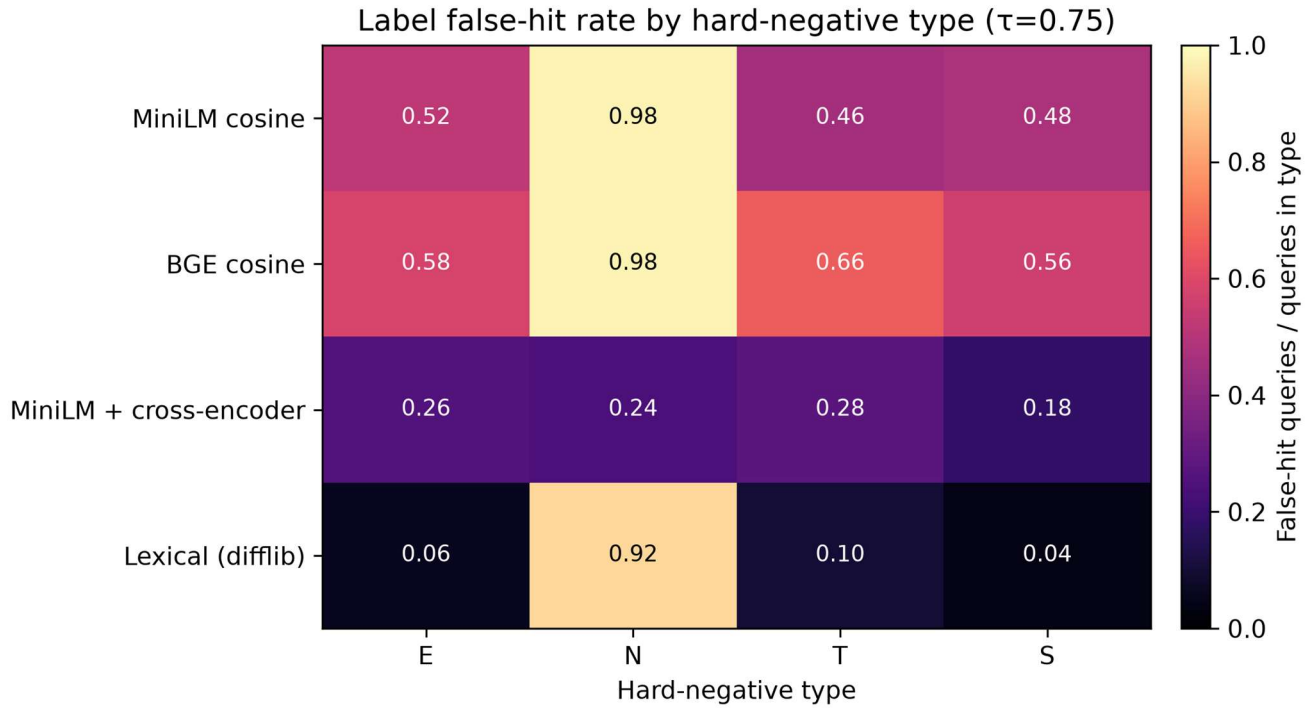


FIG. 8. Hard-negative hit rates by perturbation subtype at exactly 40% model-call savings. Each hard-negative hit is a construction-label false hit.

TABLE IX. Per-subtype JFH diagnostic and hard-negative hit rate at 40% model-call savings.

P1	JFH 25/63 (39.7%)	JFH 24/67 (35.8%)	JFH 29/72 (40.3%)	JFH 27/68 (39.7%)
P2	JFH 18/58 (31.0%)	JFH 21/63 (33.3%)	JFH 25/66 (37.9%)	JFH 21/47 (44.7%)
P3	JFH 7/27 (25.9%)	JFH 10/44 (22.7%)	JFH 18/57 (31.6%)	JFH 12/44 (27.3%)
P4	JFH 13/38 (34.2%)	JFH 13/31 (41.9%)	JFH 17/39 (43.6%)	JFH 11/21 (52.4%)
E	17/50 hit (34%); JFH 5/17 (29%)	11/50 hit (22%); JFH 1/11 (9%)	12/50 hit (24%); JFH 3/12 (25%)	18/50 hit (36%); JFH 7/18 (39%)
N	45/50 hit (90%); JFH 38/45 (84%)	40/50 hit (80%); JFH 34/40 (85%)	12/50 hit (24%); JFH 7/12 (58%)	48/50 hit (96%); JFH 41/48 (85%)
T	20/50 hit (40%); JFH 5/20 (25%)	15/50 hit (30%); JFH 3/15 (20%)	14/50 hit (28%); JFH 2/14 (14%)	17/50 hit (34%); JFH 3/17 (18%)
S	12/50 hit (24%); JFH 5/12 (42%)	9/50 hit (18%); JFH 2/9 (22%)	8/50 hit (16%); JFH 2/8 (25%)	14/50 hit (28%); JFH 5/14 (36%)
unrelated	0/100 hit; JFH NA	0/100 hit; JFH NA	0/100 hit; JFH NA	3/100 hit; JFH 3/3 (100%)

E. Lexical distance and error

Token Jaccard similarity and normalized edit distance are descriptive covariates, not calibrated proxies for semantic equivalence. At matched 40% cache calls, Table X reports paraphrase false-miss rates in bins of each measured distance. Lower Jaccard overlap and greater edit distance are associated with higher miss rates in this workload; these bins are descriptive and do not establish a causal effect.

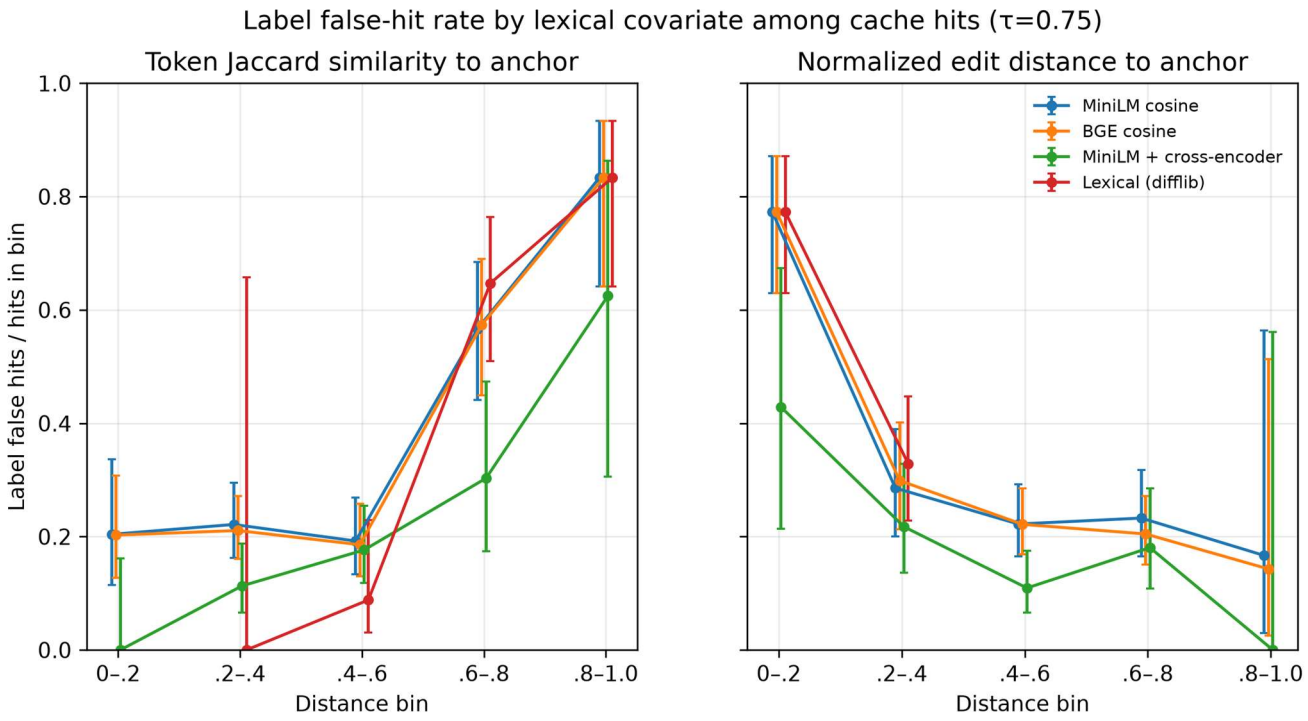


FIG. 9. Paraphrase false-miss rates by token-Jaccard and normalized edit-distance bin at exactly 40% model-call savings; labels above bars show misses / queries.

TABLE X. Paraphrase false-miss rate by level and measured lexical distance at exactly 40% model-call savings.

Paraphrase level	P1	100	37/100 (37%)	33/100 (33%)	28/100 (28%)	32/100 (32%)
Paraphrase level	P2	100	42/100 (42%)	37/100 (37%)	34/100 (34%)	53/100 (53%)
Paraphrase level	P3	100	73/100 (73%)	56/100 (56%)	43/100 (43%)	56/100 (56%)
Paraphrase level	P4	100	62/100 (62%)	69/100 (69%)	61/100 (61%)	79/100 (79%)
Token Jaccard	0.0–0.2	71	59/71 (83.1%)	57/71 (80.3%)	51/71 (71.8%)	70/71 (98.6%)
Token Jaccard	0.2–0.4	181	113/181 (62.4%)	101/181 (55.8%)	89/181 (49.2%)	119/181 (65.7%)
Token Jaccard	0.4–0.6	118	38/118 (32.2%)	37/118 (31.4%)	22/118 (18.6%)	27/118 (22.9%)
Token Jaccard	0.6–0.8	26	3/26 (11.5%)	0/26 (0.0%)	3/26 (11.5%)	4/26 (15.4%)
Token Jaccard	0.8–1.0	4	1/4 (25.0%)	0/4 (0.0%)	1/4 (25.0%)	0/4 (0.0%)
Normalized edit distance	0.0–0.2	10	1/10 (10.0%)	0/10 (0.0%)	2/10 (20.0%)	0/10 (0.0%)
Normalized edit distance	0.2–0.4	62	10/62 (16.1%)	7/62 (11.3%)	9/62 (14.5%)	0/62 (0.0%)
Normalized edit distance	0.4–0.6	163	83/163 (50.9%)	75/163 (46.0%)	51/163 (31.3%)	58/163 (35.6%)
Normalized edit distance	0.6–0.8	156	113/156 (72.4%)	106/156 (67.9%)	98/156 (62.8%)	153/156 (98.1%)
Normalized edit distance	0.8–1.0	9	7/9 (77.8%)	7/9 (77.8%)	6/9 (66.7%)	9/9 (100.0%)

F. Judge and construction-label agreement (RQ2)

The response judge fails both prespecified gates: it accepts only 70/100 (70%) anchor self-pairs, and its agreement with the audit consensus is 71.1% with Cohen's κ=0.416. Disagreement varies by stratum (Table XII). At matched 40% calls, Phi-4-mini falsely rejects 33.2–38.8% of true paraphrase hits and accepts 40.2–69.6% of label-false hits across configurations. These results answer RQ2: the stored judge verdicts do not provide a reliable primary estimate of response correctness for this workload. Table XI and Figure 10 retain JFH and judge–label disagreement as secondary diagnostics.

TABLE XI. Overall judge–label disagreement among hits at 40% model-call savings.

MiniLM cosine	104/280	37.1% (31.7%–42.9%)
BGE cosine	103/280	36.8% (31.4%–42.6%)

MiniLM + cross-encoder	121/280	43.2% (37.5%–49.1%)
Lexical (difflib)	110/280	39.3% (33.7%–45.1%)

TABLE XII. Phi-4-mini disagreement with the three-model audit consensus by stratum.

P1	10	4	40%
P2	10	4	40%
P3	10	4	40%
P4	10	1	10%
E	10	2	20%
N	10	3	30%
T	10	5	50%
S	10	3	30%
unrelated	10	0	0%

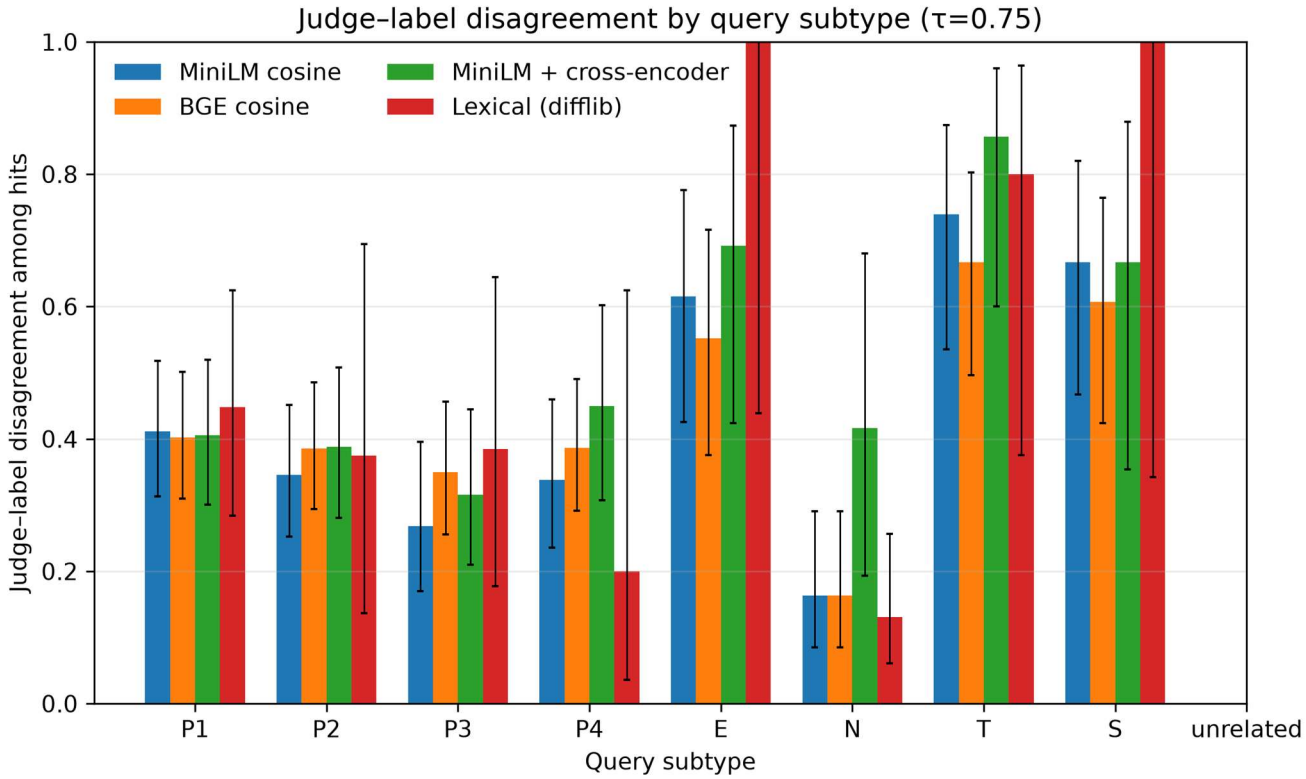


FIG. 10. Judge–label disagreement by subtype at matched 40% model-call savings; bars show Wilson 95% intervals and blank groups have no hits.

G. Relation to prior results

Published caching results use different datasets, policies and correctness definitions, so their numerical gains are not directly comparable with this controlled workload. MeanCache and SCALM report gains over their own baselines; vCache changes its policy to target a user-defined error bound, whereas our fixed-cache study maps construction-label error against call savings. The cross-encoder's JFH changes little across τ=0.50–0.90 (35.8–37.8%), but the failed judge gates show why that apparent flatness should not be interpreted as a flat cache-error curve. Table III compares these results with MT-Bench, Krites, vCache and negation analysis [4]–[7], [22], [23].

H. Implications

The equal-call comparison makes the trade-off concrete. With 40% cache calls, cross-encoder reranking has the lowest construction-label FHR (16.4%, 46/280), followed by BGE (26.8%), MiniLM (33.6%), and lexical matching (36.4%). The cross-encoder's lower hard-negative hit rates come with a 41.5% false-miss rate across

paraphrases. These results support matcher-specific evaluation of the cost–correctness trade-off; they do not establish a universal ordering or deployment-safe cutoff.

VI. Threats to Validity

Synthetic variants from one generator. Every paraphrase and hard negative comes from the same generator, a 4-billion-parameter model. The corpus may carry its stylistic habits, such as a typical sentence length or a predictable choice of words for a given perturbation, and a real user's paraphrases vary in ways one generator does not reproduce. A cache could look more or less discriminating here than on organic traffic for that reason alone. The generated phrasing may differ from naturally occurring user queries.

The audit ratings are not fully independent because Sonnet 5.5 and Claude Sonnet 4.6 are from the same model family; the raters may also share systematic biases with Phi-4-mini, so their agreement is diagnostic rather than external ground truth.

Scale and dependence. The evaluation set has at most 700 queries from 100 anchors. Queries that share an anchor are not independent, and the Wilson intervals assume they are, so they are narrower than the true uncertainty. Differences of a few points between configurations should not be treated as established. With at most 50 hard negatives per type, differences between types are visible only when they are large.

Question type. NQ-Open anchors are short factual questions drawn from search queries. Results may not carry over to long, open-ended chat prompts, where paraphrase and near-miss behave differently.

A static cache with no eviction. The cache never inserts or evicts during a run. Real caches evict under memory or time pressure, which changes which entries are available and can itself cause false misses or false hits against stale entries. Our results describe threshold and encoder behaviour in isolation from any eviction policy.

An exact index. We use an exact flat index so that no result reflects approximation error. Production systems usually use approximate indexes [12], which can miss the true nearest neighbour for reasons unrelated to the threshold. Our figures describe matching under an exact search and should be read as a best case for a given encoder and threshold.

Local, CPU-based cost. Cost is measured as model calls avoided on a CPU with small local models. Stored fallback latency describes the fallback work potentially avoided by a hit; it excludes cache lookup time and is not net serving latency. These measurements are not API prices and do not transfer directly to hosted models with their own latency and rate limits.

Judge limitations. Phi-4-mini accepts only 70% of the 100 stored anchor responses and has κ=0.416 against the stratified LLM-rating consensus. It rejects a substantial share of true paraphrase hits and accepts many label-false hits. JFH is consequently secondary in this study; future work should validate response judges on separately designed data before using judge-derived error rates as the primary outcome.

Limited set of matching methods. We compare four configurations. Fine-tuned similarity models, as in MeanCache and the Redis report, and judge-verified matching inside the policy, as in Krites and vCache, are not part of the comparison [5], [14], [7], [4]. The results characterize these four configurations under controlled variation and do not rank semantic caching approaches in general.

Single turn and English only. Every query is treated as arriving without dialogue history, which separates paraphrase effects from the context effects that ContextCache and SmartCache address [3], [15]. The results say nothing about multi-turn sessions or other languages.

Operating points chosen on evaluation data. The matched 40% point ranks the same 700 queries used to report error rates. It is a descriptive, in-sample comparison and does not estimate performance on new traffic.

VII. Conclusions

We evaluated four static-cache matchers on a controlled 700-query workload and compared them at exactly 40% cache calls using stored scores. Cross-encoder reranking had the lowest construction-label FHR (16.4%, 46/280), compared with 26.8% for BGE, 33.6% for MiniLM, and 36.4% for lexical matching. That gain came with a 41.5% paraphrase false-miss rate. At τ=0.75, the cross-encoder reduced negation hard-negative hits from 98% for the bi-encoders to 24%. These results describe one generated workload and support matcher-specific evaluation of the cost–correctness trade-off.

The study has limits that follow from its design: a static cache, single-turn English questions, a small local model stack and a modest corpus. A second generator and larger evaluation set would test whether these patterns

depend on the current workload. Eviction, approximate indexes, multi-turn queries, and response judges validated on independent data would extend the comparison to production settings.

References

[1] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474.

[2] F. Bang, "GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings," in Proc. 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS), Singapore, Dec. 2023, pp. 212–218.

[3] J. Yan, W. Ni, L. Chen, X. Lin, P. Cheng, Z. Qin, and K. Ren, "ContextCache: Context-aware semantic cache for multi-turn queries in large language models," Proc. VLDB Endowment, vol. 18, no. 12, pp. 5391–5394, 2025.

[4] L. G. Schroeder et al., "vCache: Verified semantic prompt caching," in Proc. Int. Conf. Learning Representations (ICLR), 2026, arXiv:2502.03771.

[5] W. Gill, M. Elidrisi, P. Kalapatapu, A. Ahmed, A. Anwar, and M. A. Gulzar, "MeanCache: User-centric semantic caching for LLM web services," in Proc. IEEE 39th Int. Parallel and Distributed Processing Symp. (IPDPS), 2025, arXiv:2403.02694.

[6] J. Li, C. Xu, F. Wang, I. M. von Riedemann, C. Zhang, and J. Liu, "SCALM: Towards semantic caching for automated chat services with large language models," arXiv:2406.00025, 2024.

[7] A. K. Singh, H. Wang, S. Attaluri, T. Chiam, and W. Zhu, "Asynchronous verified semantic caching for tiered LLM architectures," arXiv:2602.13165, 2026.

[8] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, Hong Kong, China, Nov. 2019, pp. 3982–3992.

[9] T. Gao, X. Yao, and D. Chen, "SimCSE: Simple contrastive learning of sentence embeddings," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2021, pp. 6894–6910.

[10] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 6769–6781.

[11] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2021.

[12] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," IEEE Trans. Pattern Anal. Mach. Intell., vol. 42, no. 4, pp. 824–836, 2020.

[13] S. Regmi and C. P. Pun, "GPT Semantic Cache: Reducing LLM costs and latency via semantic embedding caching," arXiv:2411.05276, 2024.

[14] W. Gill et al., "Advancing semantic caching for LLMs with domain-specific embeddings and synthetic data," arXiv:2504.02268, 2025.

[15] C. Yu, T. Wang, Z. Shao, and S. Jiang, "SmartCache: Context-aware semantic cache for efficient multi-turn LLM inference," in Advances in Neural Information Processing Systems (NeurIPS), 2025.

[16] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," arXiv:2305.05176, 2023.

[17] C. Jin et al., "RAGCache: Efficient knowledge caching for retrieval-augmented generation," arXiv:2404.12457, 2024.

[18] I. Gim, G. Chen, S. Lee, N. Sarda, A. Khandelwal, and L. Zhong, "Prompt Cache: Modular attention reuse for low-latency inference," arXiv:2311.04934, 2023.

[19] Y. Zhang, J. Baldridge, and L. He, "PAWS: Paraphrase adversaries from word scrambling," in Proc. NAACL-HLT, Minneapolis, MN, USA, Jun. 2019, pp. 1298–1308.

[20] M. Gardner et al., "Evaluating models' local decision boundaries via contrast sets," in Findings of the Association for Computational Linguistics: EMNLP 2020, 2020.

[21] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond accuracy: Behavioral testing of NLP models with CheckList," in Proc. 58th Annu. Meeting of the Association for Computational Linguistics (ACL), 2020.

[22] T. H. Truong, T. Baldwin, K. Verspoor, and T. Cohn, "Language models are not naysayers: An analysis of language models on negation benchmarks," in Proc. *SEM 2023, 2023.

[23] L. Zheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in Advances in Neural Information Processing Systems, vol. 36, 2023.

[24] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-Eval: NLG evaluation using GPT-4 with better alignment …," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), Singapore, Dec. 2023, pp. 2511–2522.

[25] A. Panickssery, S. R. Bowman, and S. Feng, "LLM evaluators recognize and favor their own generations," in Advances in Neural Information Processing Systems, vol. 37, 2024.

[26] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," J. Amer. Statist. Assoc., vol. 22, no. 158, pp. 209–212, 1927.

[27] L. D. Brown, T. T. Cai, and A. DasGupta, "Interval estimation for a binomial proportion," Statist. Sci., vol. 16, no. 2, pp. 101–133, 2001.

[28] K. Lee, M.-W. Chang, and K. Toutanova, "Latent retrieval for weakly supervised open domain question answering," in Proc. 57th Annu. Meeting of the Association for Computational Linguistics (ACL), Florence, Italy, Jul. 2019, pp. 6086–6096.

[29] T. Kwiatkowski et al., "Natural Questions: A benchmark for question answering research," Trans. Assoc. Comput. Linguistics, vol. 7, pp. 453–466, 2019.

[30] S. Chaudhuri, V. Ganti, and R. Kaushik, "A primitive operator for similarity joins in data cleaning," in Proc. 22nd Int. Conf. Data Engineering (ICDE), 2006.

[31] K. Lee et al., "Deduplicating training data makes language models better," in Proc. 60th Annu. Meeting of the Association for Computational Linguistics (Vol. 1: Long Papers), Dublin, Ireland, May 2022, pp. 8424–8445.

[32] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 5776–5788.

[33] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-Pack: Packed resources for general Chinese embeddings," arXiv:2309.07597, 2023.

[34] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive text embedding benchmark," in Proc. 17th Conf. European Chapter of the Association for Computational Linguistics (EACL), Dubrovnik, Croatia, May 2023, pp. 2014–2037.

[35] R. Nogueira and K. Cho, "Passage re-ranking with BERT," arXiv:1901.04085, 2019.

[36] Microsoft, "Phi-4-Mini technical report: Compact yet powerful multimodal language models via mixture-of-LoRAs," arXiv:2503.01743, 2025.

[37] Gemma Team, "Gemma 3 technical report," arXiv:2503.19786, 2025.

[38] Meta AI, “Llama 3.2 Model Card,” Meta AI, 2024. Available: https://github.com/meta-llama/llama-models/blob/main/models/llama3_2/MODEL_CARD.md.

[39] J. Cohen, “A coefficient of agreement for nominal scales,” Educational and Psychological Measurement, vol. 20, no. 1, pp. 37–46, 1960, doi: 10.1177/001316446002000104.

[40] J. R. Landis and G. G. Koch, “The measurement of observer agreement for categorical data,” Biometrics, vol. 33, no. 1, pp. 159–174, 1977, doi: 10.2307/2529310.

[41] cross-encoder, “cross-encoder/quora-distilroberta-base: Cross-Encoder for Quora Duplicate Questions Detection,” Hugging Face model card, accessed Oct. 30, 2026. Available: https://huggingface.co/cross-encoder/quora-distilroberta-base.

Appendix A. Judge prompt

Evaluate whether the cached response answers the incoming question correctly. Judge meaning and factual adequacy, not wording. Use the reference answers when present, but assess independently if they are absent. Return JSON with one key verdict whose value is exactly "correct_hit" or "false_hit".
Incoming question: {query_text!r}
Matched anchor: {anchor_question!r}
Cached response: {anchor_answer!r}
Reference answers, if available: {gold_answers!r}